

Q.1) Create a table named dmart_orders and insert values in it based on below criteria.

order_id, customer_id, customer_name, product_category, quantity, price_per_unit, order_date, payment_status.

Query:

USE JJBA;

DROP TABLE IF EXISTS dmart_orders;

CREATE TABLE dmart_orders (

order_id INT PRIMARY KEY,

customer_id VARCHAR(10),

customer_name VARCHAR(100),

product_category VARCHAR(50),

quantity INT,

price_per_unit DECIMAL(10, 2),

order_date DATE,

payment_status VARCHAR(20)

);

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL Editor window shows a series of SQL statements for creating a table and inserting data. The table 'dmart_orders' is created with columns: order_id (INT PRIMARY KEY), customer_id (VARCHAR(10)), customer_name (VARCHAR(100)), product_category (VARCHAR(50)), quantity (INT), price_per_unit (DECIMAL(10, 2)), order_date (DATE), and payment_status (VARCHAR(20)).

Below the SQL Editor, the Result Grid shows the output of the SQL statements. The first statement is 'select * from dmart_orders;', which returns 100 rows. The Result Grid displays the first 10 rows of data:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid
1003	C103	AMET MISHRA	Apparel	3	150.20	2026-01-02	Pending
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded

The bottom status bar indicates that 100 rows were returned by the query.

Q.2 Insert data of customers.

Query:

```
USE JJBA;
```

```
DROP TABLE IF EXISTS dmart_orders;
```

```
CREATE TABLE dmart_orders (
```

```
    order_id INT PRIMARY KEY,
```

```
    customer_id VARCHAR(10),
```

```
    customer_name VARCHAR(100),
```

```
    product_category VARCHAR(50),
```

```
    quantity INT,
```

```
    price_per_unit DECIMAL(10, 2),
```

```
    order_date DATE,
```

```
    payment_status VARCHAR(20)
```

```
);
```

```
INSERT INTO dmart_orders (order_id, customer_id, customer_name, product_category, quantity,  
price_per_unit, order_date, payment_status)
```

```
VALUES
```

```
(1001, 'C101', 'Rohan Sharma', 'Groceries', 5, 45.50, '2026-01-01', 'Paid'),
```

```
(1002, 'C102', 'Priya Patel', 'Electronics', 1, 1200.75, '2026-01-01', 'Paid'),
```

```
(1003, 'C103', 'AMIT MISHRA', 'Apparel', 3, 150.20, '2026-01-02', 'Pending'),
```

```
(1004, 'C101', 'Rohan Sharma', 'Packaged Foods', 10, 22.45, '2026-01-02', 'Paid'),
```

```
(1005, 'C104', 'Sneha Reddy', 'Personal Care', 2, 85.33, '2026-01-03', 'Paid'),
```

```
(1006, 'C105', 'vikram singh', 'Groceries', 12, 10.00, '2026-01-03', 'Refunded'),
```

```
(1007, 'C102', 'Priya Patel', 'Home Appliances', 1, 450.00, '2026-01-04', 'Paid'),
```

```
(1008, 'C106', 'Anjali Desai', 'Apparel', 4, 99.99, '2026-01-04', 'Pending'),
```

```
(1009, 'C103', 'AMIT MISHRA', 'Electronics', 2, 300.50, '2026-01-05', 'Paid'),
```

```
(1010, 'C107', 'Raj Malhotra', 'Groceries', 8, 35.25, '2026-01-05', 'Paid'),
```

```
(1011, 'C108', 'kiran joshi', 'Dairy', 6, 15.80, '2026-01-06', 'Paid'),
```

```
(1012, 'C104', 'Sneha Reddy', 'Packaged Foods', 5, 40.00, '2026-01-06', 'Paid'),
```

(1013, 'C109', 'Nitin Verma', 'Personal Care', 1, 120.00, '2026-01-07', 'Pending'),

(1014, 'C101', 'Rohan Sharma', 'Groceries', 2, 65.40, '2026-01-07', 'Paid'),

(1015, 'C110', 'Pooja Hegde', 'Apparel', 5, 110.00, '2026-01-08', 'Paid'),

(1016, 'C105', 'vikram singh', 'Electronics', 1, 899.00, '2026-01-08', 'Paid'),

(1017, 'C106', 'Anjali Desai', 'Home Appliances', 2, 175.25, '2026-01-09', 'Refunded'),

(1018, 'C111', 'Sunil Shetty', 'Groceries', 20, 5.50, '2026-01-09', 'Paid'),

(1019, 'C102', 'Priya Patel', 'Personal Care', 3, 45.00, '2026-01-10', 'Paid'),

(1020, 'C107', 'Raj Malhotra', 'Apparel', 1, 250.00, '2026-01-10', 'Pending'),

(1021, 'C112', 'Meera Nair', 'Packaged Foods', 15, 12.20, '2026-01-11', 'Paid'),

(1022, 'C108', 'kiran joshi', 'Groceries', 4, 55.00, '2026-01-11', 'Paid'),

(1023, 'C113', 'deepak Gupta', 'Electronics', 1, 650.00, '2026-01-12', 'Paid'),

(1024, 'C109', 'Nitin Verma', 'Dairy', 10, 18.00, '2026-01-12', 'Paid'),

(1025, 'C103', 'AMIT MISHRA', 'Groceries', 7, 42.10, '2026-01-13', 'Paid'),

(1026, 'C114', 'Ritu Phogat', 'Personal Care', 2, 95.50, '2026-01-13', 'Paid'),

(1027, 'C110', 'Pooja Hegde', 'Packaged Foods', 8, 25.00, '2026-01-14', 'Refunded'),

(1028, 'C101', 'Rohan Sharma', 'Home Appliances', 1, 520.00, '2026-01-14', 'Paid'),

(1029, 'C115', 'Gaurav Kapur', 'Apparel', 3, 180.00, '2026-01-15', 'Paid'),

(1030, 'C106', 'Anjali Desai', 'Groceries', 11, 14.50, '2026-01-15', 'Pending'),

(1031, 'C112', 'Meera Nair', 'Dairy', 4, 22.00, '2026-01-16', 'Paid'),

(1032, 'C116', 'Arjun Kapoor', 'Electronics', 2, 410.00, '2026-01-16', 'Paid'),

(1033, 'C105', 'vikram singh', 'Packaged Foods', 6, 30.25, '2026-01-17', 'Paid'),

(1034, 'C113', 'deepak Gupta', 'Personal Care', 4, 60.00, '2026-01-17', 'Paid'),

(1035, 'C107', 'Raj Malhotra', 'Groceries', 9, 38.00, '2026-01-18', 'Paid'),

(1036, 'C117', 'Neha Kakkar', 'Apparel', 2, 299.99, '2026-01-18', 'Paid'),

(1037, 'C108', 'kiran joshi', 'Home Appliances', 1, 340.00, '2026-01-19', 'Refunded'),

(1038, 'C114', 'Ritu Phogat', 'Groceries', 5, 48.00, '2026-01-19', 'Paid'),

(1039, 'C118', 'Varun Dhawan', 'Packaged Foods', 12, 15.00, '2026-01-20', 'Pending'),

(1040, 'C102', 'Priya Patel', 'Dairy', 3, 28.50, '2026-01-20', 'Paid'),

(1041, 'C115', 'Gaurav Kapur', 'Electronics', 1, 999.00, '2026-01-21', 'Paid'),

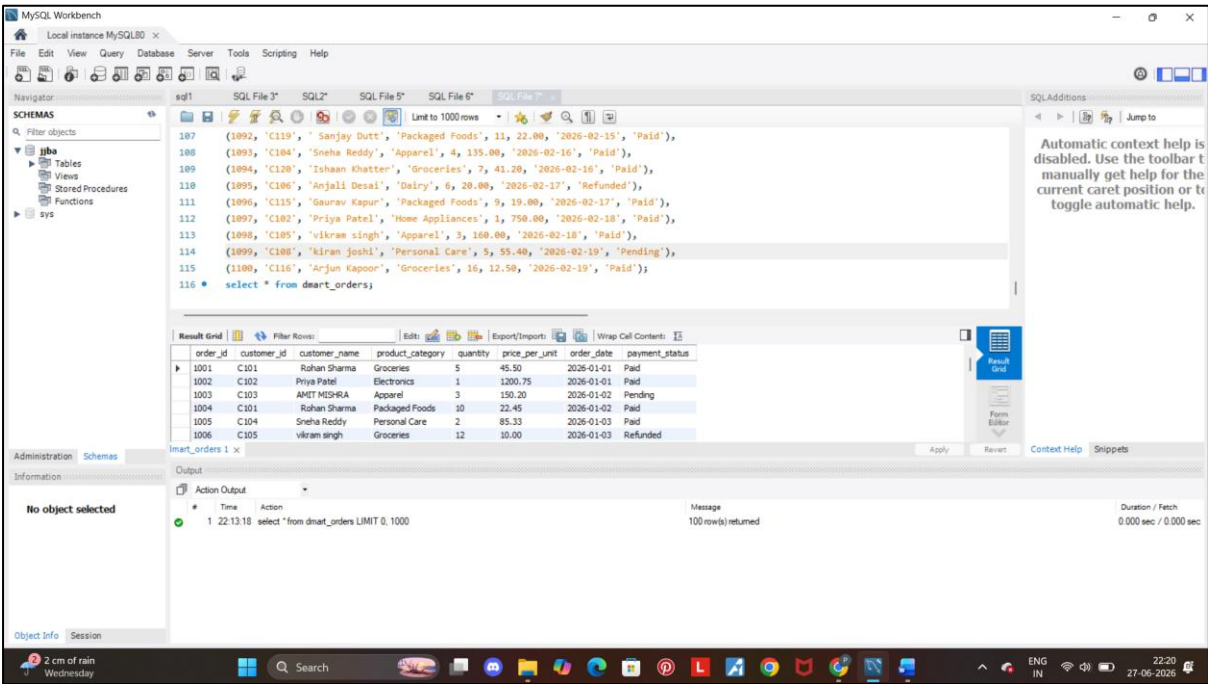
(1042, 'C110', 'Pooja Hegde', 'Personal Care', 3, 75.20, '2026-01-21', 'Paid'),

(1043, 'C119', 'Sanjay Dutt', 'Groceries', 14, 12.00, '2026-01-22', 'Paid'),

(1044, 'C103', 'AMIT MISHRA', 'Packaged Foods', 4, 32.50, '2026-01-22', 'Paid'),
(1045, 'C106', 'Anjali Desai', 'Electronics', 1, 150.00, '2026-01-23', 'Paid'),
(1046, 'C120', 'Ishaan Khatter', 'Apparel', 6, 85.00, '2026-01-23', 'Pending'),
(1047, 'C112', 'Meera Nair', 'Home Appliances', 1, 680.00, '2026-01-24', 'Paid'),
(1048, 'C109', 'Nitin Verma', 'Groceries', 10, 21.00, '2026-01-24', 'Paid'),
(1049, 'C116', 'Arjun Kapoor', 'Personal Care', 2, 110.50, '2026-01-25', 'Refunded'),
(1050, 'C113', 'deepak Gupta', 'Packaged Foods', 7, 24.00, '2026-01-25', 'Paid'),
(1051, 'C101', 'Rohan Sharma', 'Electronics', 1, 1350.00, '2026-01-26', 'Paid'),
(1052, 'C117', 'Neha Kakkar', 'Groceries', 6, 52.30, '2026-01-26', 'Paid'),
(1053, 'C104', 'Sneha Reddy', 'Dairy', 5, 19.50, '2026-01-27', 'Paid'),
(1054, 'C118', 'Varun Dhawan', 'Apparel', 3, 195.00, '2026-01-27', 'Paid'),
(1055, 'C108', 'kiran joshi', 'Packaged Foods', 9, 18.50, '2026-01-28', 'Pending'),
(1056, 'C115', 'Gaurav Kapur', 'Home Appliances', 2, 210.00, '2026-01-28', 'Paid'),
(1057, 'C105', 'vikram singh', 'Personal Care', 4, 65.00, '2026-01-29', 'Paid'),
(1058, 'C119', 'Sanjay Dutt', 'Electronics', 1, 720.00, '2026-01-29', 'Paid'),
(1059, 'C102', 'Priya Patel', 'Groceries', 13, 11.20, '2026-01-30', 'Refunded'),
(1060, 'C120', 'Ishaan Khatter', 'Packaged Foods', 5, 33.00, '2026-01-30', 'Paid'),
(1061, 'C107', 'Raj Malhotra', 'Dairy', 8, 16.00, '2026-01-31', 'Paid'),
(1062, 'C114', 'Ritu Phogat', 'Apparel', 2, 320.00, '2026-01-31', 'Paid'),
(1063, 'C116', 'Arjun Kapoor', 'Groceries', 15, 9.50, '2026-02-01', 'Pending'),
(1064, 'C110', 'Pooja Hegde', 'Home Appliances', 1, 890.00, '2026-02-01', 'Paid'),
(1065, 'C103', 'AMIT MISHRA', 'Personal Care', 3, 88.00, '2026-02-02', 'Paid'),
(1066, 'C113', 'deepak Gupta', 'Groceries', 7, 44.00, '2026-02-02', 'Paid'),
(1067, 'C117', 'Neha Kakkar', 'Packaged Foods', 10, 20.00, '2026-02-03', 'Paid'),
(1068, 'C109', 'Nitin Verma', 'Apparel', 4, 140.00, '2026-02-03', 'Refunded'),
(1069, 'C112', 'Meera Nair', 'Electronics', 1, 540.00, '2026-02-04', 'Paid'),
(1070, 'C118', 'Varun Dhawan', 'Groceries', 11, 16.50, '2026-02-04', 'Paid'),
(1071, 'C101', 'Rohan Sharma', 'Dairy', 6, 21.50, '2026-02-05', 'Paid'),
(1072, 'C119', 'Sanjay Dutt', 'Home Appliances', 1, 430.00, '2026-02-05', 'Pending'),
(1073, 'C104', 'Sneha Reddy', 'Groceries', 8, 39.00, '2026-02-06', 'Paid'),
(1074, 'C120', 'Ishaan Khatter', 'Personal Care', 2, 105.00, '2026-02-06', 'Paid'),

(1075, 'C106', 'Anjali Desai', 'Packaged Foods', 14, 14.00, '2026-02-07', 'Paid'),
 (1076, 'C115', 'Gaurav Kapur', 'Groceries', 5, 58.20, '2026-02-07', 'Paid'),
 (1077, 'C102', 'Priya Patel', 'Apparel', 3, 210.50, '2026-02-08', 'Refunded'),
 (1078, 'C105', 'vikram singh', 'Dairy', 4, 24.00, '2026-02-08', 'Paid'),
 (1079, 'C108', 'kiran joshi', 'Electronics', 1, 1150.00, '2026-02-09', 'Paid'),
 (1080, 'C116', 'Arjun Kapoor', 'Packaged Foods', 8, 26.50, '2026-02-09', 'Paid'),
 (1081, 'C107', 'Raj Malhotra', 'Home Appliances', 2, 199.00, '2026-02-10', 'Pending'),
 (1082, 'C114', 'Ritu Phogat', 'Electronics', 1, 850.00, '2026-02-10', 'Paid'),
 (1083, 'C111', 'Sunil Shetty', 'Packaged Foods', 25, 10.50, '2026-02-11', 'Paid'),
 (1084, 'C110', 'Pooja Hegde', 'Groceries', 12, 18.00, '2026-02-11', 'Paid'),
 (1085, 'C103', 'AMIT MISHRA', 'Dairy', 5, 23.00, '2026-02-12', 'Paid'),
 (1086, 'C113', 'deepak Gupta', 'Apparel', 2, 185.00, '2026-02-12', 'Refunded'),
 (1087, 'C117', 'Neha Kakkar', 'Personal Care', 4, 72.00, '2026-02-13', 'Paid'),
 (1088, 'C109', 'Nitin Verma', 'Home Appliances', 1, 620.00, '2026-02-13', 'Paid'),
 (1089, 'C112', 'Meera Nair', 'Groceries', 10, 33.50, '2026-02-14', 'Paid'),
 (1090, 'C118', 'Varun Dhawan', 'Electronics', 1, 499.00, '2026-02-14', 'Pending'),
 (1091, 'C101', 'Rohan Sharma', 'Personal Care', 3, 90.00, '2026-02-15', 'Paid'),
 (1092, 'C119', 'Sanjay Dutt', 'Packaged Foods', 11, 22.00, '2026-02-15', 'Paid'),
 (1093, 'C104', 'Sneha Reddy', 'Apparel', 4, 135.00, '2026-02-16', 'Paid'),
 (1094, 'C120', 'Ishaan Khatter', 'Groceries', 7, 41.20, '2026-02-16', 'Paid'),
 (1095, 'C106', 'Anjali Desai', 'Dairy', 6, 20.00, '2026-02-17', 'Refunded'),
 (1096, 'C115', 'Gaurav Kapur', 'Packaged Foods', 9, 19.00, '2026-02-17', 'Paid'),
 (1097, 'C102', 'Priya Patel', 'Home Appliances', 1, 750.00, '2026-02-18', 'Paid'),
 (1098, 'C105', 'vikram singh', 'Apparel', 3, 160.00, '2026-02-18', 'Paid'),
 (1099, 'C108', 'kiran joshi', 'Personal Care', 5, 55.40, '2026-02-19', 'Pending'),
 (1100, 'C116', 'Arjun Kapoor', 'Groceries', 16, 12.50, '2026-02-19', 'Paid');
 select * from dmart_orders;

Screenshot:



Q. 3 Calculate the total price of products and create a separate column for it.

Query:

```
select *,(price_per_unit*quantity) as total_price from dmart_orders;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor at the top contains the following query:

```
select *,(price_per_unit*quantity) as total_price from dmart_orders;
```

Below the editor, the 'Result Grid' shows the output of the query. The results are as follows:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	total_price
1001	C101	Rohan Sharma	Groceries	5	45.50	2026-01-01	Paid	227.50
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	1200.75
1003	C103	AMIT MISHRA	Apparel	3	150.20	2026-01-02	Pending	450.60
1004	C101	Rohan Sharma	Packaged Foods	10	22.45	2026-01-02	Paid	224.50
1005	C104	Sneha Reddy	Personal Care	2	85.33	2026-01-03	Paid	170.66
1006	C105	Vikram Singh	Groceries	12	10.00	2026-01-03	Refunded	120.00

The bottom panel shows the 'Action Output' tab, which displays the execution of the query and the resulting error messages. The first message indicates that 100 rows were affected. Subsequent messages show 'Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1' for the same query, suggesting a syntax error in the original query provided in the text.

Insights:

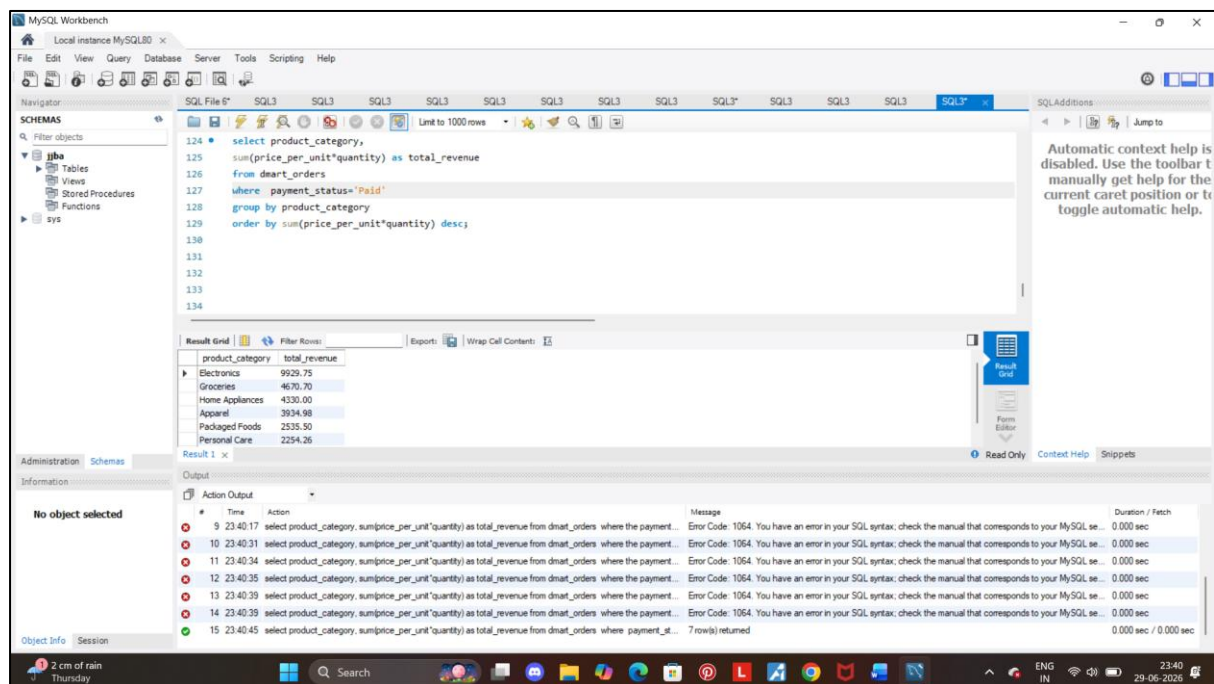
- The price_per_unit depicts the base price of each item.
- The total_price column is derived by multiplying the quantity and its price per unit.

Q. 4 Calculate to total revenue generated across all the product categories.

Query:

```
select product_category,  
  
sum(price_per_unit*quantity) as total_revenue  
  
from dmart_orders  
  
where payment_status='Paid'  
  
group by product_category  
  
order by sum (price_per_unit*quantity) desc;
```

Screenshot:



Insights:

- The total revenue generated across all the categories over the duration of 2 months is calculated.
- Where customers spend 9929 rupees on 'Electronic' and the least spend category is 'Dairy' with 1013 rupees of spend, generating the highest revenue for the store.

Q. 5 Which product categories are lagging in pending status.

Query:

```
select product_category,  
count(order_id) as total_pending_orders,  
sum(quantity) as total_pending_quantity  
from dmart_orders  
where payment_status='Pending'  
group by product_category  
order by count(order_id) desc;
```

Screenshot:

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
121 (sum(case when payment_status='Paid' then price_per_unit*quantity else 0 end)/sum(price_per_unit*quantity))*100,2),'%')  
122 as revenue_ratio  
123 from dmart_orders;"  
124 • select product_category,  
125 count(order_id) as total_pending_orders,  
126 sum(quantity) as total_pending_quantity  
127 from dmart_orders  
128 where payment_status='Pending'  
129 group by product_category  
130 order by count(order_id) desc;
```

The Results Grid shows the following data:

product_category	total_pending_orders	total_pending_quantity
Apparel	4	14
Personal Care	2	6
Groceries	2	26
Packaged Foods	2	21
Home Appliances	2	3
Electronics	1	1

The Output pane shows the following messages:

```
28 23:58:33 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount whe... Error Code: 1054. Unknown column 'product_category' in 'field list' 0.015 sec  
29 23:58:57 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount from... Error Code: 1054. Unknown column 'total_pending_quantity' in 'order clause' 0.000 sec  
30 23:58:58 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount from... Error Code: 1054. Unknown column 'total_pending_quantity' in 'order clause' 0.000 sec  
31 23:59:10 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount from... Error Code: 1054. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se... 0.000 sec  
32 23:59:19 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount from... Error Code: 1054. Unknown column 'total_pending_quantity' in 'order clause' 0.000 sec  
33 23:59:37 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_amount from... 6 row(s) returned 0.000 sec / 0.000 sec  
34 00:00:54 select product_category, count(order_id) as total_pending_orders, sum(quantity) as total_pending_quantity from... 6 row(s) returned 0.000 sec / 0.000 sec
```

Insights:

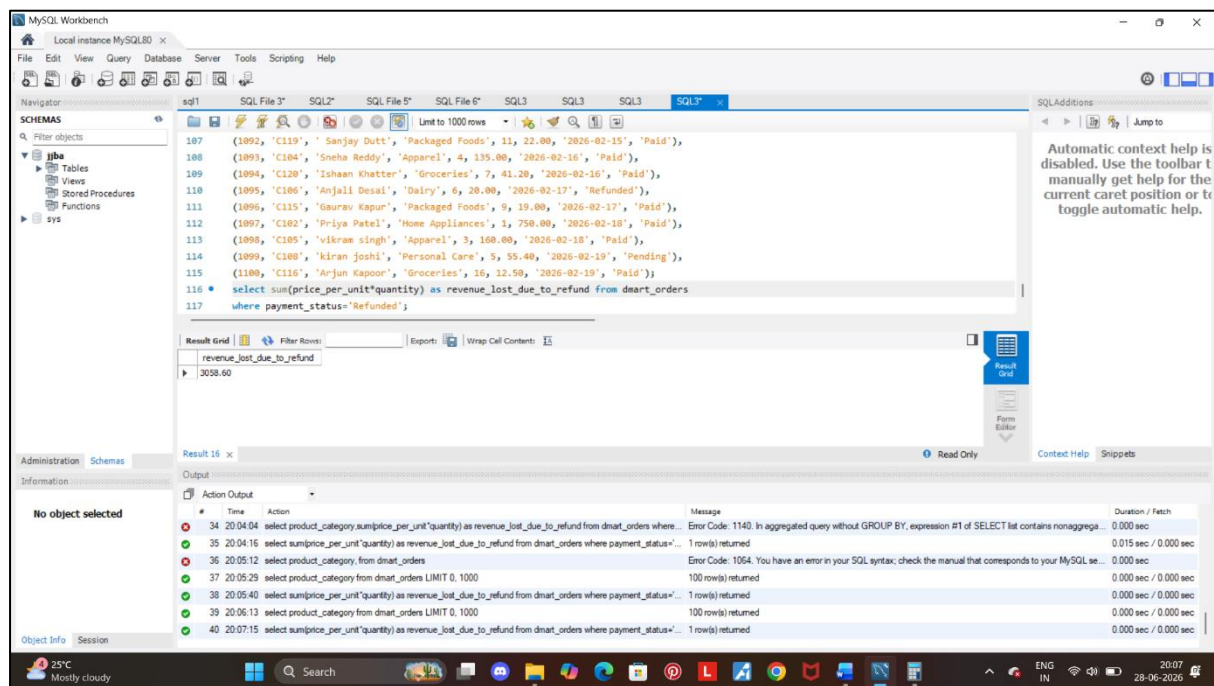
- Apparel has 4 'Pending' orders being the highest in count.
- While the Package Foods has only two orders but the order quantity is 21. This concludes that when it comes to 'Package Foods' category most of the bulk buyer are either delaying the payment or the payment process is stuck.

Q. 6 Calculate the total revenue lost due to refund transaction.

Query:

```
select sum(price_per_unit*quantity) as revenue_lost_due_to_refund from dmart_orders  
where payment_status='Refunded';
```

Screenshot:



Insights:

- The total revenue lost due refund is 3058.
- The digit depicts that approx 9% of money is refunded among the total orders placed including 'Paid', 'refunded' and 'pending' orders as well which were 35710.

Q. 7 Print the details of top 10 customers with highest total spend.

Query:

```
select *,(price_per_unit*quantity) as total_revenue from dmart_orders order by total_revenue desc limit 10;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor at the top contains the following query:

```
select *,(price_per_unit*quantity) as total_revenue from dmart_orders order by total_revenue desc limit 10;
```

Below the editor, the 'Result Grid' shows the top 10 orders by total revenue. The results are as follows:

order_id	customer_id	customer_name	product_category	quantity	price_per_unit	order_date	payment_status	total_revenue
1001	C101	Rohan Sharma	Electronics	1	1350.00	2026-01-26	Paid	1350.00
1002	C102	Priya Patel	Electronics	1	1200.75	2026-01-01	Paid	1200.75
1079	C108	Kiran Joshi	Electronics	1	1150.00	2026-02-09	Paid	1150.00
1041	C115	Gaurav Kapur	Electronics	1	999.00	2026-01-21	Paid	999.00
1016	C105	Vikram Singh	Electronics	1	899.00	2026-01-08	Paid	899.00
1064	C110	Pooja Hegde	Home Appliances	1	890.00	2026-02-01	Paid	890.00

The bottom section of the screenshot shows the 'Action Output' pane, which lists the execution steps and their durations. The final step shows that 10 rows were returned for the query.

Insights:

- Most of the 'Paid' and most shopped category is electronic with generated total revenue of 19,728 rupees.
- 'Home Appliances' being the second most profitable category with sales of 1,640 rupees.

Q. 8 Calculate the total items sold across all transactions.

Query:

```
select payment_status, sum(quantity) as total_items_sold from dmart_orders group by payment_status;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor at the top contains the following query:

```
select payment_status, sum(quantity) as total_items_sold from dmart_orders group by payment_status;
```

Below the editor, the 'Result Grid' shows the output of the query:

payment_status	total_items_sold
Paid	420
Pending	71
Refunded	53

The bottom panel shows the 'Action Output' and 'Messages' sections. The 'Messages' section contains several error messages, including 'Error Code: 1055. Expression #3 of SELECT list is not in GROUP BY clause and contains nonaggregated column...' and 'Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggregated column...'. The 'Action Output' section shows the execution of the query and the resulting rows.

Insights:

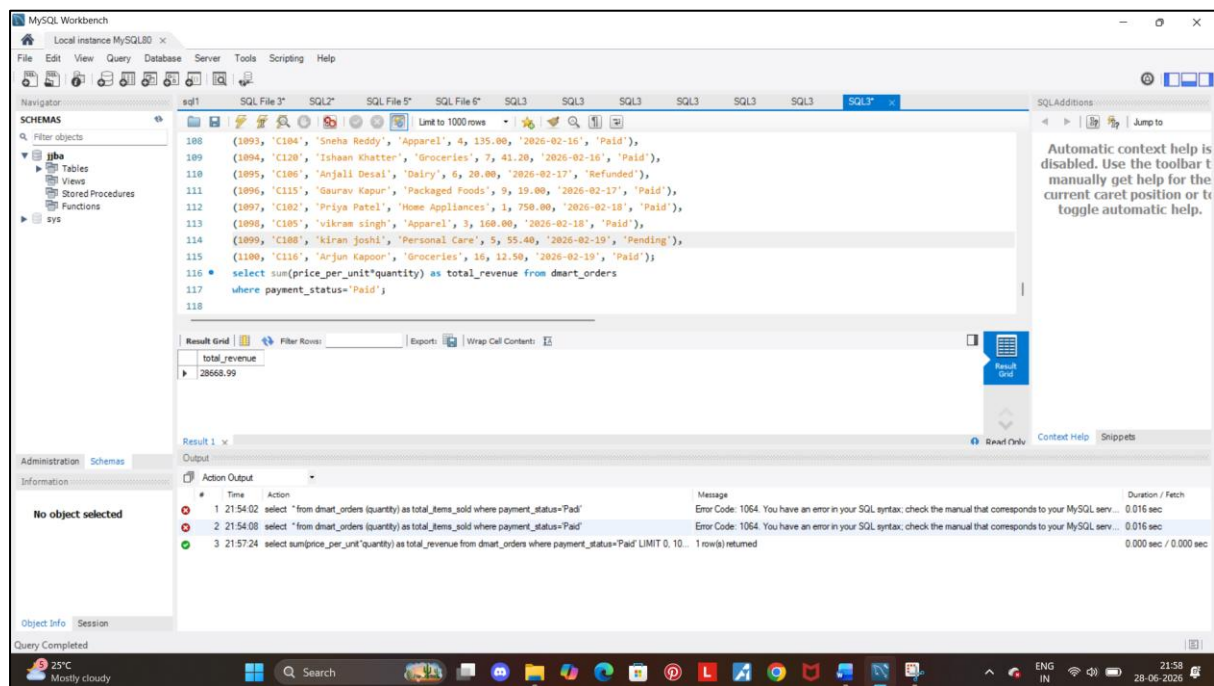
- The sum of quantities sold across 3 different transactions are calculated.
- The total orders placed across all the transactions are 544.
- The 'Pending' section covering the 13% of it, 'Refund' covering 8% and 'Paid' section with 77% of it.
- The pending transaction status among users is higher, which may mean customers facing problems in transaction.

Q. 9 Total revenue generated across all the paid orders.

Query:

```
select sum(price_per_unit*quantity) as total_revenue from dmart_orders  
where payment_status='Paid';
```

Screenshot:



Insights:

- The total revenue generated 28,668 for all the successfully processed payments.
- There is an additional potential revenue that has not been included and is currently sitting as 'Pending' and 'Refund'.

Q. 10 Calculate the average revenue generated per single unit item sold.

Query:

```
select product_category,  
  
round((sum(price_per_unit*quantity)/sum(quantity)),2)  
  
as revenue_generated_per_unit_sold  
  
from dmart_orders  
  
where payment_status='Paid'  
  
group by product_category  
  
order by revenue_generated_per_unit_sold desc;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor contains a query that calculates the average revenue generated per unit sold for each product category, filtered by 'Paid' payment status. The query is as follows:

```
(sum(case when payment_status='Paid' then price_per_unit*quantity else 0 end)/sum(price_per_unit*quantity))*100,2),'%')  
as revenue_ratio  
from dmart_orders;*/  
select product_category,  
round((sum(price_per_unit*quantity)/sum(quantity)),2)  
as revenue_generated_per_unit_sold  
from dmart_orders  
where payment_status='Paid'  
group by product_category  
order by revenue_generated_per_unit_sold desc;
```

The Results Grid shows the output of the query:

product_category	revenue_generated_per_unit_sold
Electronics	709.22
Home Appliances	541.25
Apparel	178.86
Personal Care	75.14
Groceries	28.14

The bottom panel shows the Action Output and Message logs, indicating that the query was executed successfully and returned 7 rows.

Insights:

- The total revenue generated per unit item sold is the highest for 'Electronic' sitting at 709 with most paid payments.
- While the 'Packaged Food' being at lowest with most least cleared transactions.

Q. 11 Which category has the highest loss due to refund orders .

Query:

```
select product_category,  
  
sum(price_per_unit*quantity) as revenue_lost  
  
from dmart_orders  
  
where payment_status='Refunded'  
  
group by product_category  
  
order by revenue_lost desc;
```

Screenshot:

The screenshot displays the MySQL Workbench interface. The SQL editor contains a query to calculate the revenue lost for refunded orders, grouped by product category. The results are shown in a table with two columns: product_category and revenue_lost. The categories are ranked from highest to lowest revenue lost.

product_category	revenue_lost
Apparel	1561.50
Home Appliances	696.50
Groceries	265.60
Personal Care	221.00
Packaged Foods	200.00

The bottom panel shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
73	16:08:36	select product_category, sum(quantity) as total_quantity, avg(sum(quantity)) as average_item_bought from dm...	Error Code: 1111. Invalid use of group function	0.000 sec
74	16:09:25	select product_category, avg(quantity) as average_item_bought from dmart_orders where payment_status='Pat...	7 row(s) returned	0.016 sec / 0.000 sec
75	16:11:16	select product_category, round(avg(quantity),2) as average_item_bought from dmart_orders where payment_st...	7 row(s) returned	0.000 sec / 0.000 sec
76	16:22:50	select product_category, sum(price_per_unit*quantity) as revenue_lost from dmart_orders where payment_stat...	Error Code: 1054. Unknown column 'price_per_unit' in field list	0.000 sec
77	16:22:57	select product_category, sum(price_per_unit*quantity) as revenue_lost from dmart_orders where payment_statu...	6 row(s) returned	0.000 sec / 0.000 sec
78	16:24:34	select product_category, sum(price_per_unit*quantity) as revenue_lost from dmart_orders where payment_statu...	1 row(s) returned	0.000 sec / 0.000 sec
79	16:24:40	select product_category, sum(price_per_unit*quantity) as revenue_lost from dmart_orders where payment_statu...	6 row(s) returned	0.000 sec / 0.000 sec

Insights:

- The 'Apparel' category is at the top with the potential revenue loss of 1,561 rupees.
- With 'Dairy' being at the lowest with loss of 120 rupees, indicating the return rate of customers in dairy category is significantly less.

Q. 12 Which product category is mostly bought in bulk.

Query:

```
select product_category,  
round(avg(quantity),2) as average_item_bought  
from dmart_orders  
where payment_status='Paid'  
group by product_category  
order by average_item_bought desc;
```

Screenshot:

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that concatenates a revenue ratio with the average item bought per category. The query is as follows:

```
148 concat(  
149 round((sum(case when payment_status='Paid' then price_per_unit*quantity else 0 end)/sum(price_per_unit*quantity))*100,2),'%'  
150 as revenue_ratio  
151 from dmart_orders;*/  
152 select product_category,  
153 round(avg(quantity),2) as average_item_bought  
154 from dmart_orders  
155 where payment_status='Paid'  
156 group by product_category  
157 order by average_item_bought desc;
```

The Results Grid shows the following data:

product_category	average_item_bought
Packaged Foods	9.92
Groceries	8.74
Dairy	5.67
Apparel	3.14
Personal Care	3.00

The Output tab shows the execution log with several error messages (Error Code: 1111. Invalid use of group function) and a successful query execution (7 rows returned).

Insights:

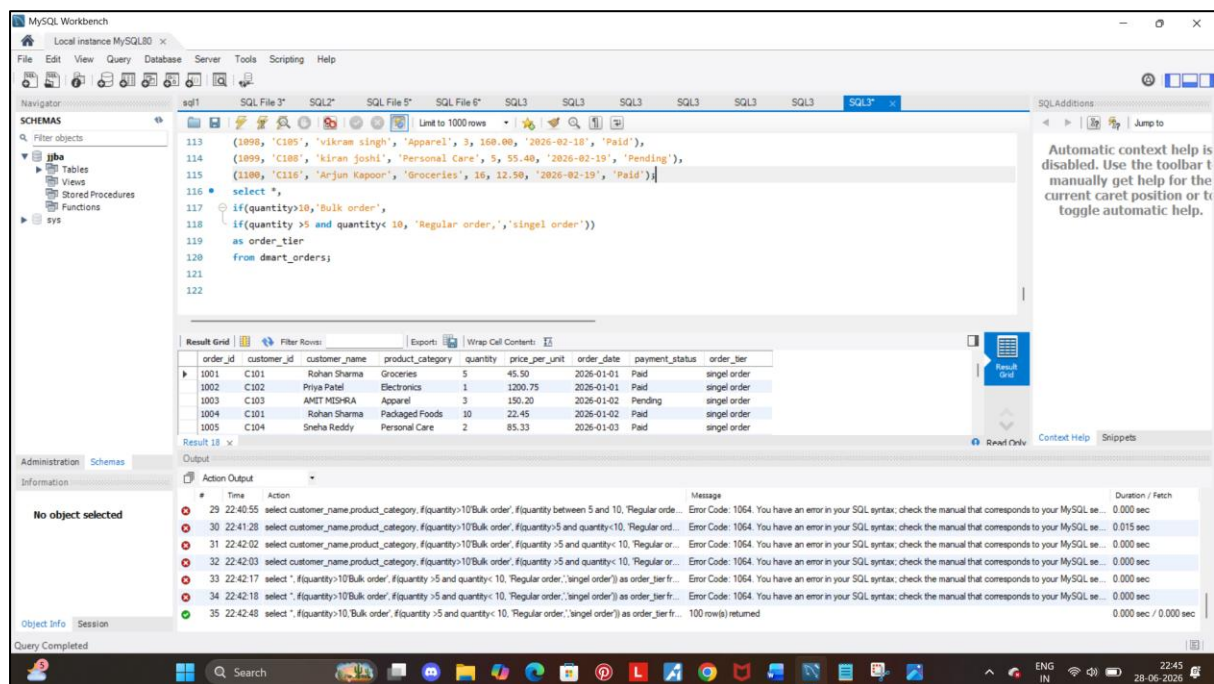
- 'Package Food' category tend to have more bulk buyers with 9 items per cart
- With 'Electronic' and 'Home Appliances' at the bottom with 1-2 items per cart, preferred as a single purchase among customers.

Q. 13 Create a new column order_tier labelling each customer as per the order quantity.

Query:

```
select *,
if(quantity>10,'Bulk order',
if(quantity >5 and quantity< 10, 'Regular order','singel order'))
as order_tier
from dmart_orders;
```

Screenshot:



Insights:

- Based on the quantity of products bought by the customers, the most common order_tier being 'single order', and the second most being the 'regular'.
- 'Groceries' and 'Packaged Food' being the two categories with most of the bulk buyers.